Parameter Passing

Hangil Kim*

June 20, 2026

Abstract

Put broadly, parameter passing is a mechanism for sending data to procedures. There are many ways to achieve this goal that produce different program semantics. We procedurally discuss various parameter passing strategies in the context of a compact imperative language with lexical blocks.[1]

Contents

1 Introduction 1

2 Syntax 2

3 Strategies 3
- 3.1 Call-By-Value . . . 4
- 3.2 Call-By-Reference . . . 8
- 3.3 Call-By-Name . . . 9
- 3.4 Call-By-Need . . . 11

4 Comparison 13

A Static Typing 14

1 Introduction

Parameter passing is a mechanism for sending data to procedures. There are many ways to achieve this goal that produce different program semantics—we will focus on the most commonplace approaches.

We first provide the syntax of a small imperative language named Base (Section 2). We then consider call-by-value (Section 3.1), call-by-reference (Section 3.2), call-by-name (Section 3.3), and call-by-need (Section 3.4) variants of that language. As a summary, we provide a relatively succinct program in which each strategy produces a different result (Section 4). We also define a type system for the language (Appendix A); the parameter passing strategy does not affect the type system whatsoever, so it resides in an appendix.

In this discussion, we assume the reader is familiar with the following.

*kimhang@oregonstate.edu

[1] A complete implementation can be found at https://github.com/hshkk/Base.

- Syntax presented as grammars. We use a slightly modified variant of Backus-Naur form in this article.
- Evaluation (and typing) rules presented as inference rule systems.

Related Works

H. R. Nielson and F. Nielson [1992] provide a more extensive treatment of a similar language named **Proc**, focusing on its operational, denotational, and axiomatic semantics. The present work instead shifts focus toward the interplay of different parameter passing strategies. Still, the semantics of Base deals with similar mechanisms as those presented by H. R. Nielson and F. Nielson [1992, Section 4.5].

2 Syntax

To discuss parameter passing strategies, we define a relatively limited imperative language named Base with lexical blocks. In Figure 1, we provide the syntax for Base.

	x, y, f	$\in$	Var	
	c	$\in$	Con	
	Π	$\in$	Prim	
	$\oplus$	$\in$	Bin	
Prog	P	$::=$	$\{s;\ \ldots\ ;\ s;\}$	
Stmt	s	$::=$	$x : \Pi$	*(declaration)*
		$\mid$	$x = e$	*(assignment)*
		$\mid$	**proc** $f(\overline{x : \Pi})Q$	*(procedure declaration)*
		$\mid$	P	*(lexical block)*
Body	Q	$::=$	$\{s;\ \ldots\ ;\ s;$ **return** $e;\}$	*(procedure body)*
Expr	e	$::=$	x	*(variable)*
		$\mid$	c	*(constant)*
		$\mid$	$e \oplus e$	*(binary operation)*
		$\mid$	$f(\overline{e})$	*(procedure invocation)*

Figure 1: The syntax for Base programs. Programs consist of lexical blocks of statements. Statements are either declarations assigned to primitive types (`Int`, `Bool`, etc.), assignments, or procedure definitions (we reserve the metavariable f for procedure names). We stipulate that procedures always return an expression, incorporating a metavariable Q for procedure bodies to force this requirement. Return statements can only reside at the outermost level of a body. For parameter/argument lists, we use a bar to succinctly represent a comma-separated sequence.

Base is not a particularly interesting language as it lacks constructs such as conditionals, loops, or higher-order functions. As will soon be developed, its evaluation rules will also not admit recursion. Nevertheless, the intention is not for it to be usable in a practical setting, but for it to lend towards a relatively minimal rule system.

	ℓ	$\in$	Loc	*(memory locations;* Int*)*
Val	v	$::=$	c	*(constant)*
		$\mid$	$(\rho, (\overline{x})Q)$	*(closure)*
Env	ρ	$::=$	$\bullet$	*(empty environment)*
		$\mid$	$\rho, x \to \ell$	*(extend with "x points to ℓ")*

Figure 2: The value and environment language for Base programs.

For concreteness, we provide the following example Base program that incorporates many of its syntactic elements. For the sake of brevity, we will admit the notation $x : \Pi = e$ to indicate simultaneous declaration and assignment.

```
{   a : Int;
    b : Int = 5;
    {   proc f(x : Int, y : Int){
            z : Int = x;
            return z + y;
        };
        a = f(1, b);
    };
    {   proc g(x : Bool, y : Bool){
            x = x && y;
            proc h(z : Bool){
                a = 10 * 2;
                return z && y;
            };
            return h(x);
        };
    };
}
```

Figure 2 gives the value and environment language for Base programs. As will soon be developed, procedures will evaluate to a *closure*, capturing a snapshot of the variables accessible at the place of its definition. In the closure $(\rho, (\overline{x})Q)$, ρ is the environment at the time of a procedure's declaration, while $\overline{x}$ and Q are its parameters and body respectively. This snapshotting allows Base to be statically scoped.

Environments map variables to locations, which can be viewed as a generalization of memory addresses. To distinguish them from integer constants, we will write locations in bold (e.g., **1**).

3 Strategies

Broadly, the goal of evaluation is to reduce a "thing" into some irreducible "thing". In traditional settings, this means reducing expressions down into values. For instance, an arithmetic expression reduces to a numeric value, and a Boolean expression reduces to a Boolean value. The Base language is structured around programs at the uppermost level, so we must first identify what it means for a *program* to evaluate.

Programs consist of statements, which define variables at runtime by assigning expressions to them. Over the course of a program's execution, this set of variables (as

well as the values they are assigned to) can change. We can therefore view program evaluation as producing *state transformations*: a program maps an initial state to a final state consisting of variable bindings.[2] Parameter passing is intimately connected to this view, as the mechanism by which arguments are bound to parameters is itself a form of state construction. Each strategy determines precisely how and when those bindings are introduced into the state.

Under this viewpoint, evaluation of programs (or statements) should take a state and yields another state. Expressions reasonably evaluate to values, but they might also affect the state. For instance, the consider the following program.

```
1  {   a : Int = 5;
2      proc f(x : Int){
3          a = 3;
4          return x;
5      }
6      r : Int = f(a);
7  }
```

When `f` is invoked by `r` at line `6`, the value of `a` is modified inside its body. This modification must then be reflected in the scope `r` lies within. The evaluation of expressions (or procedure bodies) consequently takes a state and produces both a value and another state.

The "statefulness" of imperative evaluation contrasts with the evaluation of (pure) functional languages, where expressions do not modify the state. There, evaluation is often modeled by a judgment that maps a state and expression directly to a value.

Notation

The bar notation is convenient for succinctly denoting comma-separated sequences, but leaves ambiguity when enforcing that multiple sequences share the same length. For example, if a procedure f has n parameters, then any invocation of f must supply exactly n arguments.[3] To enforce arity constraints, we write $(A_i)^{i \in 1..n}$ to denote the sequence $A_1, \ldots, A_n$ for some A; this notation extends straightforwardly to judgments.

3.1 Call-By-Value

Call-by-value (CBV) is one of the most common parameter passing strategies. Under CBV, any expression passed into a procedure as an argument is first fully evaluated, then bound to a corresponding variable. Frequently, this is achieved in implementation by coping the value into a new place in memory—we will achieve this here by literally copying the values of the arguments into fresh locations, removing them from the state once the invocation has been fully evaluated. To distinguish the semantic variants of Base we will discuss, we name the CBV variant *cbv*-Base.

[2] Due to this indirection, imperative programs are often more difficult to formally reason about in comparison to pure functional programs. Correspondingly, rule systems for imperative languages tend to be more complex.

[3] For simplicity, we do not allow partial procedure application.

$$\begin{array}{llll} \text{Mem} & \mu & ::= & \bullet \quad \textit{(empty memory store)} \\ & & \mid & \mu, \ell{=}v \quad \textit{(extend with “}\ell \textit{ is } v\textit{”)} \end{array}$$

Figure 3: The representation of stores in memory for *cbv*-Base programs.

In Figure 3, we provide a representation of a store in memory for *cbv*-Base programs. Whereas environments map variables to locations, stores map locations to values [H. R. Nielson and F. Nielson 1992, Section 4.5].

Operations on Environments and Memory Stores

Before proceeding further, we provide a brief explanation on several operations that will prove to be useful in the discussion of semantics. We use μ as the basis for these definitions, but they can be straightforwardly extended to act on ρ.

The *domain* of a map is the elements on “the left”: $dom(\mu) = \{\ell : \ell{=}v \in \mu\}$. The *range* instead pulls from “the right”: $ran(\mu) = \{v : \ell{=}v \in \mu\}$. We denote *substitution* in maps as $\mu[\ell{=}v]$, which extends μ with the binding $\ell{=}v$. If $\ell \notin dom(\mu)$, the new binding is added to the map rather than substituted. We also admit the notation $\mu|_L$ to *restrict* μ only to locations also in L. For example, if we define $\mu_1 = \mathbf{1}{=}1, \mathbf{2}{=}3$ and $\mu_2 = \mathbf{1}{=}5$, then $\mu_1|_{dom(\mu_2)} = \mathbf{1}{=}1$.

Finally, we write $x{=}v \in \mu \circ \rho$ to succinctly denote the predicate $x{\to}\ell \in \rho \wedge \ell{=}v \in \mu$.

Operational Semantics

Figure 4 gives the semantics of *cbv*-Base. Rule *cbv*-Var looks up the location of a variable in the environment and produces the value stored there. Rules *cbv*-Con and *cbv*-Bin are straightforward. Rule *cbv*-Ret evaluates a procedure body by first evaluating each constituent statement in sequence, then evaluating the expression inside **`return`**.

Procedure invocation. Rule *cbv*-Inv embodies the essence of CBV. Its first premise affirms that the invoked procedure resolves to a closure in memory. Each argument is then evaluated in sequence, threading the memory store to yield a final store μ_n. After a set of fresh locations are generated, the procedure body is evaluated with 1) the environment extended to map parameter names to those locations, and 2) μ_n extended with the corresponding argument values. The result is the value produced by the body, together with the resultant memory store restricted to the domain of the original store μ_0, thereby discarding all fresh locations.

Declaration and assignment. Variable declarations extend the environment with a fresh location, while assignments update existing locations with new values. Procedure declarations simultaneously extend both the environment and the store by creating a fresh location mapped to a closure.

Nested blocks. Rule *cbv*-Internal evaluates a nested block but discards the resultant environment, forgetting any new bindings introduced within it. The resultant memory store may nevertheless contain updates to variables declared before the nested block, and is therefore restricted to $ran(\rho)$. This raises a question: why use $ran(\rho)$ rather than $dom(\mu)$, given that both describe sets of locations?

$\boxed{\rho, \mu \vdash _ \Downarrow v, \mu'}$ *(an expression/body evaluates to v in ρ and μ, producing μ')*

$$\frac{x{=}v \in \mu \circ \rho}{\rho, \mu \vdash x \Downarrow v, \mu}\ \textit{cbv}\text{-Var} \qquad \frac{}{\rho, \mu \vdash c \Downarrow c, \mu}\ \textit{cbv}\text{-Con}$$

$$\frac{\rho, \mu_0 \vdash e_1 \Downarrow v_1, \mu_1 \qquad \rho_0, \mu_1 \vdash e_2 \Downarrow v_2, \mu_2 \qquad v_1 \oplus v_2 = v}{\rho, \mu_0 \vdash e_1 \oplus e_2 \Downarrow v, \mu_2}\ \textit{cbv}\text{-Bin}$$

$$\frac{\begin{array}{l} f{=}(\rho', (x_1, \ldots, x_n)Q) \in \mu \circ \rho \qquad (\rho, \mu_{i-1} \vdash e_i \Downarrow v_i, \mu_i)^{i \in 1..n} \\ (\ell_i \text{ fresh in } \rho)^{i \in 1..n} \qquad \rho'[(x_i{\to}\ell_i)^{i \in 1..n}], \mu_n[(\ell_i{=}v_i)^{i \in 1..n}] \vdash Q \Downarrow v, \mu \end{array}}{\rho, \mu_0 \vdash f(e_1, \ldots, e_n) \Downarrow v, \mu|_{dom(\mu_0)}}\ \textit{cbv}\text{-Inv}$$

$$\frac{(\rho_{i-1}, \mu_{i-1} \vdash s_i \dashv \rho_i, \mu_i)^{i \in 1..n} \qquad \rho_n, \mu_n \vdash e \Downarrow v, \mu}{\rho_0, \mu_0 \vdash \{s_1\texttt{;}\ \ldots\ \texttt{;}\ s_n\texttt{;}\ \textbf{return}\ e\texttt{;}\} \Downarrow v, \mu}\ \textit{cbv}\text{-Ret}$$

$\boxed{\rho, \mu \vdash _ \dashv \rho', \mu'}$ *(a statement/program transforms ρ and μ into ρ' and μ')*

$$\frac{\ell \text{ fresh in } \rho}{\rho, \mu \vdash x : \Pi \dashv \rho[x{\to}\ell], \mu}\ \textit{cbv}\text{-Decl} \qquad \frac{x{\to}\ell \in \rho \qquad \rho, \mu \vdash e \Downarrow v, \mu'}{\rho, \mu \vdash x = e \dashv \rho, \mu'[\ell{=}v]}\ \textit{cbv}\text{-Assign}$$

$$\frac{\ell \text{ fresh in } \rho}{\rho, \mu \vdash \textbf{proc}\ f(\overline{x : \Pi})Q \dashv \rho[f{\to}\ell], \mu[\ell{=}(\rho, (\overline{x : \Pi})Q)]}\ \textit{cbv}\text{-Proc}$$

$$\frac{\rho, \mu \vdash P \dashv \rho', \mu'}{\rho, \mu \vdash P \dashv \rho, \mu'|_{ran(\rho)}}\ \textit{cbv}\text{-Internal} \qquad \frac{(\rho_{i-1}, \mu_{i-1} \vdash s_i \dashv \rho_i, \mu_i)^{i \in 1..n}}{\rho_0, \mu_0 \vdash \{s_1\texttt{;}\ \ldots\ \texttt{;}\ s_n\texttt{;}\} \dashv \rho_n, \mu_n}\ \textit{cbv}\text{-Prog}$$

Figure 4: The semantics of *cbv*-Base, the call-by-*value* variant of Base. In rule *cbv*-Internal, we restrict the domain of μ' to prevent internal bindings from leaking out of nested blocks and to update any preexisting bindings. In rule *cbv*-Inv we restrict to remove all parameter bindings but preserve updates made to other variables.

When a variable is declared, its location is added to the environment but no value is assigned in the memory store.[4] If a nested block then assigns to that variable, the assignment is recorded in the store returned from the block. Restricting that store by $dom(\mu)$ would discard the assignment, since the location was absent from the store before the block was evaluated. Restricting by $ran(\rho)$ is correct, as it retains all locations that the nested block could reach, regardless of whether they had been assigned to prior to the block.

As a concrete example, consider the following program, in which `y` is declared and later assigned inside a nested block.

```
1  {   x : Int;
2      y : Int;
3      {   z : Int;
4          y = 7;
5      };
6  }
```

Immediately after line 2, we have $\rho = \texttt{x}{\to}\textbf{1}, \texttt{y}{\to}\textbf{2}$ and $\mu = \bullet$, since neither variable has been assigned. Evaluation of the nested block produces $\rho' = \texttt{x}{\to}\textbf{1}, \texttt{y}{\to}\textbf{2}, \texttt{z}{\to}\textbf{3}$ and $\mu' = \textbf{2}{=}7$. Since $\textbf{2} \notin dom(\mu)$, restricting μ' by $dom(\mu)$ would yield $\bullet$, losing the assignment to `y`. Restricting by $ran(\rho)$ correctly preserves **2**, since `y` remains in scope in the enclosing environment. Since `z` is declared inside the nested block, returning ρ rather than ρ' as the resultant environment correctly discharges that binding.

Conversely, we could imagine restricting μ by $ran(\rho)$ in rule *cbv*-Inv. However, this also produces unexpected behavior. Consider the following program.

```
1  {   proc f(){
2          return 1;
3      };
4      proc g(){
5          return f();
6      };
7      r : Int;
8      r = g();
9  }
```

Immediately following line 7, $\rho = \texttt{f}{\to}\textbf{1}, \texttt{g}{\to}\textbf{2}, \texttt{r}{\to}\textbf{3}$ and $\mu = \textbf{1}{=}(\bullet, \texttt{()}\ldots), \textbf{2}{=}(\texttt{f}{\to}\textbf{1}, \texttt{()}\ldots)$. We now consider the derivation tree for the evaluation of `g()` at line 8, maintaining those definitions.

$$\dfrac{\texttt{g}{=}(\texttt{f}{\to}\textbf{1}, \texttt{()}\ldots) \in \mu \circ \rho \qquad \dfrac{\texttt{f}{=}(\bullet, \texttt{()}\ldots) \in \mu \circ (\texttt{f}{\to}\textbf{1}) \qquad \dfrac{\dfrac{}{\bullet, \mu \vdash \texttt{1} \Downarrow \texttt{1}, \mu}}{\bullet, \mu \vdash \texttt{\{return 1;\}} \Downarrow \texttt{1}, \mu}}{(\texttt{f}{\to}\textbf{1}), \mu \vdash \texttt{f()} \Downarrow \texttt{1}, \mu|_{dom(\mu)} (= \mu)}}{(\texttt{f}{\to}\textbf{1}), \mu \vdash \texttt{\{return f();\}} \Downarrow \texttt{1}, \mu}}{\rho, \mu \vdash \texttt{g()} \Downarrow \texttt{1}, \mu|_{dom(\mu)} (= \mu)}$$

The shaded judgment is precisely where the usage of $ran(\rho)$ would produce unexpected behavior. At that moment, $\rho = \texttt{f}{\to}\textbf{1}$, meaning that $\mu|_{ran(\rho)}$ would contain

[4] This distinction is why fresh locations are generated with respect to ρ and not μ in Figure 4.

$\boxed{\rho, \mu \vdash _ \Downarrow v, \mu'}$ *(an expression/body evaluates to v in ρ and μ, producing μ')*

$$\frac{f{=}(\rho', (y_1, \ldots, y_n)Q) \in \mu \circ \rho \quad (x_i{\to}\ell_i \in \rho)^{i\in 1..n} \quad \rho'[(y_i{\to}\ell_i)^{i\in 1..n}], \mu \vdash Q \Downarrow v, \mu'}{\rho, \mu \vdash f(x_1, \ldots, x_n) \Downarrow v, \mu'|_{dom(\mu)}}\ \textit{cbr}\text{-Inv}$$

$$\textit{cbr}\text{-}\mathfrak{F} = \textit{cbv}\text{-}\mathfrak{F} \quad \forall \mathfrak{F} \in \{\text{Var}, \text{Con}, \ldots\}$$

Figure 5: The semantics of *cbr*-Base, the call-by-*reference* variant of Base. The rule system as a whole is very much similar to that depicted in Figure 4 for *cbv*-Base, so we have omitted rules that would only clutter the presentation, using the metavariable $\mathfrak{F}$ to range over the remaining rule names. The rule for procedure invocation replaces the corresponding rule in the CBV system.

just **1**. The disappearance of **2** would then propagate all the way down the tree (the path of which is represented with arrows), meaning g would suddenly be without a corresponding value in the resultant store.

In summary, nested blocks restrict by the *range of the environment* to preserve locations declared but not yet assigned. Procedure invocation restricts by the *domain of the store* since it must preserve locations that exist in memory at the moment of invocation, but are absent from environments captured in closures.

3.2 Call-By-Reference

Call-by-reference (CBR) is another common parameter passing strategy in imperative languages that have indirection mechanisms like pointers. Under CBR, procedures receive an alias (i.e., a location) of the actual argument, rather than a copy of a value like under CBV. Any changes made to the alias are treated as changes to the original variable.

The primary benefit of CBR is that it avoids duplicating potentially large values, since any change made to the alias will flow back to the caller, eliminating the need to reserve additional space in memory. We model this indirection here by pointing variables in the environment to the same location as others. Hence, the definition of memory stores in *cbr*-Base is fully inherited from *cbv*-Base (Figure 3). A further restriction arises from the semantics of CBR: the mechanism only makes sense when the caller passes a variable (rather than an arbitrary expression), so we restrict procedure arguments in *cbr*-Base accordingly.

$$\text{Expr} \quad e \quad ::= \quad \ldots \mid f(\overline{x}) \quad \textit{(procedure invocation under CBR)}$$

Figure 5, gives the semantics of *cbr*-Base. To avoid repetition, we only explicitly write out the single rule that departs from the semantics of *cbv*-Base.

Rule *cbr*-Inv departs from its CBV equivalent by avoiding evaluation of the arguments. The first premise of rule *cbn*-Inv affirms that the invoked procedure resolves to

$$
\begin{array}{llll}
\text{Mem} & \mu & ::= & \bullet & \textit{(empty memory store)} \\
& & \mid & \mu, \ell{=}v & \textit{(extend with “}\ell \textit{ is } v\textit{”)} \\
& & \mid & \mu, \ell{=}\langle \rho, e \rangle & \textit{(extend with “}\ell \textit{ is } \langle \rho, e \rangle\textit{”)}
\end{array}
$$

Figure 6: The representation of stores in memory for *cbn*-Base programs. In addition to the preexisting definition (for *cbv*-Base and *cbr*-Base; Figure 3) we incorporate thunks, which are pairs of environments and expressions.

a closure in memory. After the locations pointed to by the arguments are retrieved, the procedure body is evaluated with the environment extended to map parameter names to those locations. The result is the value produced by the body, together with the resultant memory store. Unlike in *cbv*-Base, there is no need to restrict the store, since no fresh locations have been generated.

3.3 Call-By-Name

Similar to CBR, *call-by-name* (CBN) is a parameter passing strategy in which arguments are not evaluated when they are bound to corresponding parameters. However, the restriction that all parameters be variables in *cbr*-Base does not apply to *cbn*-Base (or *cbNeed*-Base, introduced in Section 3.4). Instead, locations are bound to entire *expressions*, not just values. Since expressions can contain variables, we also package it with the environment at the point of its binding; this environment-expression pair is called a *thunk*.

The evaluation of a thunk is deferred until the corresponding value is needed. Thus, CBN can avoid unnecessary work if some of the arguments are not used at all in the procedure, as in the following program where a parameter `y` of a procedure `f` is never used.

```
1  {   proc f(x : Int, y : Int){
2          return x;
3      };
4      r : Int = f(2, f(3+7,4)-2);
5  }
```

CBV would fully evaluate the argument `f(3+7,4)-2` before proceeding, but this would only take up unnecessary computational resources. CBN defers evaluation until it is actually needed, saving the machine from doing more work than is necessary.

Operational Semantics

Figure 7 gives the semantics of *cbn*-Base, again omitting rules that do not depart from the semantics of *cbv*-Base.

Procedure invocation. The first premise of rule *cbn*-Inv affirms that the invoked procedure resolves to a closure in memory. After a set of fresh locations are generated, the procedure body is evaluated with 1) the environment extended to map parameter names to those locations, and 2) the memory store extended with the corresponding

$\boxed{\rho, \mu \vdash _ \Downarrow v, \mu'}$ *(an expression/body evaluates to v in ρ and μ, producing μ')*

$$\frac{x{=}\langle \rho', e\rangle \in \mu \circ \rho \qquad \rho', \mu \vdash e \Downarrow v, \mu'}{\rho, \mu \vdash x \Downarrow v, \mu} \; \textit{cbn}\text{-Force}$$

$$\frac{f{=}(\rho', (x_1, \ldots, x_n)Q) \in \mu \circ \rho \qquad (\ell_i \text{ fresh in } \rho)^{i\in 1..n} \\ \rho'[(x_i{\to}\ell_i)^{i\in 1..n}], \mu[(\ell_i{=}\langle \rho, e_i\rangle)^{i\in 1..n}] \vdash Q \Downarrow v, \mu}{\rho, \mu \vdash f(e_1, \ldots, e_n) \Downarrow v, \mu'|_{dom(\mu)}} \; \textit{cbn}\text{-Inv}$$

$$\textit{cbn}\text{-}\mathfrak{J} = \textit{cbv}\text{-}\mathfrak{J} \quad \forall \mathfrak{J} \in \{\text{Var}, \text{Con}, \ldots\}$$

Figure 7: The semantics of *cbn*-Base, the call-by-*name* variant of Base. The rule system as a whole is very much similar to that depicted in Figure 4 for *cbv*-Base, so we have omitted rules that would only clutter the presentation, using the metavariable $\mathfrak{J}$ to range over the remaining rule names. The rule for procedure invocation replaces the corresponding rule in the CBV system.

argument *thunks*. The result is the value produced by the body, together with the resultant memory store restricted to the domain of the original store, thereby discarding all fresh locations.

Forcing thunks. Rule *cbn*-Force is a completely new rule. It states that if a variable points to a thunk in memory, then its evaluation must be forced to produce a value. The expression of a thunk is evaluated in the packaged environment, producing a value and an (ignored) memory store.

Very little languages use CBN in practice, since it can be deeply inefficient: *cbn*-Force does not cache the evaluation result (the memory store is returned as is), so every time a variable bound to an expression gets evaluated, the associated expression must be evaluated again. It can also lead to surprising results: the values that variables point to within thunk expressions may evolve over time, meaning that a forced thunk can produce a value based on information unavailable at the time of its creation.

As an example, consider the following program.

```
1  {   a : Int = 1;
2      b : Int = 1;
3      proc f(c : Int){
4          d : Int = c;
5          b = a + 3;
6          return c + d;
7      };
8      r : Int = f(a+b);
9  }
```

Immediately after binding the parameter `c` following the invocation of `f` at line 8, we have $\rho = \texttt{a}{\to}\textbf{1}, \texttt{b}{\to}\textbf{2}, \texttt{c}{\to}\textbf{4}$ and $\mu = \textbf{1}{=}1, \textbf{2}{=}1, \textbf{3}{=}(\ldots, (\texttt{c})\ldots), \textbf{4}{=}\langle(\texttt{a}{\to}\textbf{1}, \texttt{b}{\to}\textbf{2}, \texttt{f}{\to}\textbf{3}), \texttt{a+b}\rangle$. For brevity, we name the environment stored in the thunk ρ'. First, we consider the

derivation tree for the evaluation of just the assignment `d = c` at line 4.

$$\dfrac{\mathtt{d}\to\mathbf{5} \in \rho \qquad \dfrac{\mathtt{c}=\langle \rho', \mathtt{a+b}\rangle \in \mu \circ \rho \qquad \dfrac{\dfrac{\vdots}{\rho', \mu \vdash \mathtt{b} \Downarrow 1, \mu} \quad \ldots}{\rho', \mu \vdash \mathtt{a+b} \Downarrow 7, \mu} \quad \ldots}{\rho, \mu \vdash \mathtt{c} \Downarrow 2, \mu}}{\rho, \mu \vdash \mathtt{d = c} \dashv \rho, (\mu, \mathbf{5}{=}2)}$$

(define $\rho = (\mathtt{a}\to\mathbf{1}, \mathtt{b}\to\mathbf{2}, \mathtt{c}\to\mathbf{4}, \mathtt{d}\to\mathbf{5})$ and $\mu = (\mathbf{1}{=}1, \mathbf{2}{=}1, \mathbf{3}{=}(\ldots, (\mathtt{c})\ldots), \mathbf{4}{=}\langle \rho', \mathtt{a+b}\rangle)$)

Evaluating `d` forces the thunk corresponding to `c`, giving 2 as a result. Next, we consider the derivation tree for the returned expression `c+d`.

$$\dfrac{\dfrac{\mathtt{c}=\langle \rho', \mathtt{a+b}\rangle \in \mu \circ \rho \qquad \dfrac{\dfrac{\vdots}{\rho', \mu \vdash \mathtt{b} \Downarrow 4, \mu} \quad \ldots}{\rho', \mu \vdash \mathtt{a+b} \Downarrow 7, \mu}}{\rho, \mu \vdash \mathtt{c} \Downarrow 7, \mu} \qquad \dfrac{\mathtt{d}{=}2 \in \mu \circ \rho}{\rho, \mu \vdash \mathtt{d} \Downarrow 2, \mu} \qquad 7 + 2 = 9}{\rho, \mu \vdash \mathtt{c+d} \Downarrow 9, \mu}$$

(define $\rho = (\mathtt{a}\to\mathbf{1}, \mathtt{b}\to\mathbf{2}, \mathtt{c}\to\mathbf{4}, \mathtt{d}\to\mathbf{5})$ and $\mu = (\mathbf{1}{=}1, \mathbf{2}{=}4, \mathbf{3}{=}(\ldots, (\mathtt{c})\ldots), \mathbf{4}{=}\langle \rho', \mathtt{a+b}\rangle, \mathbf{5}{=}2)$)

Forcing `c` again produces a different result, 7, due to `b` having been assigned $\mathtt{a+3} = 4$ in line 5, as evidenced by the shaded judgments. Thus, `c+d` gives the result 9. Since Base permits arbitrary assignment as a side effect, the resulting impact under CBN is that thunks may be affected by changes to the memory store as program execution proceeds.

3.4 Call-By-Need

Call-by-need (CBNeed) ameliorates the issue of CBN by caching evaluation results to be reused throughout a program. Now, not only do we avoid evaluating expressions that might not need to be, but we also avoid unnecessarily reevaluating an expression bound to a variable multiple times. Otherwise, the definition of memory stores in *cbNeed*-Base is identical to the one for *cbn*-Base (Figure 6).

In Figure 8 we provide the semantics of *cbNeed*-Base. The only meaningfully different rule from *cbn*-Base is rule *cbNeed*-Force, which evaluates a thunk pointed to by a variable, then *updates the memory store with the resultant value.* Any future occurrence of that variable will then go through rule *cbNeed*-Var in evaluation, which is a simple lookup and thus more efficient in general.

Note that while CBN and CBNeed are closely related (differing only in the store produced after forcing a thunk), their behavior is not generally the same. In particular, they differ in a language with side effects; Base allows assignment, which is a side effect. Again, we consider the example program from Section 3.3.

$\boxed{\rho, \mu \vdash _ \Downarrow v, \mu'}$ *(an expression/body evaluates to v in ρ and μ, producing μ')*

$$\frac{x{\to}\ell \in \rho \qquad \ell{=}\langle \rho', e\rangle \in \mu \qquad \rho' \vdash e \Downarrow v, \mu'}{\rho, \mu \vdash x \Downarrow v, \mu[\ell{=}v]}\ \textit{cbNeed}\text{-Force}$$

$$\textit{cbNeed}\text{-}\mathfrak{I} = \textit{cbn}\text{-}\mathfrak{I} \quad \forall \mathfrak{I} \in \{\text{Var}, \text{Con}, \ldots\}$$

Figure 8: The semantics of *cbNeed*-Base, the call-by-*need* variant of Base. The rule system as a whole is very much similar to that depicted in Figure 7 for *cbn*-Base, so we have omitted rules that would only clutter the presentation, using the metavariable $\mathfrak{I}$ to range over the remaining rule names. The rule for procedure invocation replaces the corresponding rule in the CBN system.

```
1  {   a : Int = 1;
2      b : Int = 1;
3      proc f(c : Int){
4          d : Int = c;
5          b = a + 3;
6          return c + d;
7      };
8      r : Int = f(a+b);
9  }
```

As before, we first consider the derivation tree for the assignment to `d` at line `4`, where $\rho' = \texttt{a}{\to}\textbf{1}, \texttt{b}{\to}\textbf{2}, \texttt{f}{\to}\textbf{3}$.

$$\dfrac{\texttt{d}{\to}\textbf{5} \in \rho \qquad \dfrac{\texttt{c}{=}\langle \rho', \texttt{a+b}\rangle \in \mu \circ \rho \qquad \dfrac{\ldots \quad \dfrac{\vdots}{\rho', \mu \vdash \texttt{b} \Downarrow 1, \mu} \quad \ldots}{\rho', \mu \vdash \texttt{a+b} \Downarrow 7, \mu}}{\rho, \mu \vdash \texttt{c} \Downarrow 2, \mu[\textbf{4}{=}2]}}{\rho, \mu \vdash \texttt{d} = \texttt{c} \dashv \rho, (\mu[\textbf{4}{=}2], \textbf{5}{=}2)}$$

(define $\rho = (\texttt{a}{\to}\textbf{1}, \texttt{b}{\to}\textbf{2}, \texttt{c}{\to}\textbf{4}, \texttt{d}{\to}\textbf{5})$ and $\mu = (\textbf{1}{=}1, \textbf{2}{=}1, \textbf{3}{=}(\ldots, (\texttt{c})\ldots), \textbf{4}{=}\langle \rho', \texttt{a+b}\rangle)$)

Evaluating `d` forces the thunk corresponding to `c`, giving `2` as a result *and an updated store binding* **4** *to* 2 *as well.* Next, we consider the derivation tree for the returned expression `c+d`, emphasizing that **4** no longer points to a thunk, but a *value.*

$$\dfrac{\dfrac{\texttt{c}{=}2 \in \mu \circ \rho}{\rho, \mu \vdash \texttt{c} \Downarrow 2, \mu} \qquad \dfrac{\texttt{d}{=}2 \in \mu \circ \rho}{\rho, \mu \vdash \texttt{d} \Downarrow 2, \mu} \qquad 2 + 2 = 4}{\rho, \mu \vdash \texttt{c+d} \Downarrow 4, \mu}$$

(define $\rho = (\texttt{a}{\to}\textbf{1}, \texttt{b}{\to}\textbf{2}, \texttt{c}{\to}\textbf{4}, \texttt{d}{\to}\textbf{5})$ and $\mu = (\textbf{1}{=}1, \textbf{2}{=}4, \textbf{3}{=}(\ldots, (\texttt{c})\ldots), \textbf{4}{=}2, \textbf{5}{=}2)$)

As evidenced by the shaded judgment, the evaluation of `c` is now a simple lookup in ρ and μ, giving `2`. The evaluation of `c+d` thus produces `4` as a result.

Line	Call-by-value	Call-by-reference	Call-by-name	Call-by-need
1	a=1	a=1	a=1	a=1
9	a=1 f=(a=1, (x)...)	a=1 f=(a=1, (x)...)	a=1 f=(a=1, (x)...)	a=1 f=(a=1, (x)...)
10↪2	a=1, x=1	a=1, x→a	a=1, x=a	a=1, x=a
3	a=2, x=1	a=2, x→a	a=2, x=a	a=2, x=a
4	a=2, x=1, b=1	a=2, x→a, b=2	a=2, x=a, b=2	a=2, x=2, b=2
5	a=3, x=1, b=1	a=3, x→a, b=2	a=3, x=a, b=2	a=3, x=2, b=2
6	a=3, x=1, b=1, c=1	a=3, x→a, b=2, c=3	a=3, x=a, b=2, c=3	a=3, x=2, b=2, c=2
7	a=3, x=2, b=1, c=1	a=4, x→a, b=2, c=3	a=3, x=4, b=2, c=3	a=3, x=3, b=2, c=2
8↪10	a=3, f=..., r=4	a=4, f=..., r=9	a=3, f=..., r=9	a=3, f=..., r=7
11	a=3, f=..., r=7	a=4, f=..., r=13	a=3, f=..., r=12	a=3, f=..., r=10

Figure 9: A trace of the state at each point of execution of the program provided in Section 4 under each parameter passing strategy. The notation $n \rightarrow m$ present in the **Line** column is meant to illustrate a transitory movement from line n to line m, particularly when binding parameters or returning from procedure bodies.

4 Comparison

We claim that each semantic variant of Base assigns different values to r at the end of following program.

```
1   {   a : Int = 1;
2       proc f(x : Int){
3           a = a + 1;
4           b : Int = x;
5           a = a + 1;
6           c : Int = x;
7           x = x + 1;
8           return b + c + x;
9       };
10      r : Int = f(a);
11      r = r + a;
12  }
```

We provide the trace of the state at each point of execution in Figure 9. For brevity, we have combined the environment with the memory state, using $x \rightarrow y$ to make it clear when a variable points to the same location as another.

The first meaningful difference in evaluation is when the parameter x is bound at line 10↪2. Under CBV, the argument a is immediately evaluated, producing 1, and bound to x. Under CBR, x is instead made to point to a. CBN and CBNeed both create a thunk from x to a.

The next difference is on line 4. Under CBV, the incrementing of a on the previous line does not affect the value of x, so b is bound to 1. On the other hand, the other three variants assign b to 2, since the change to a is picked up by each. Under CBNeed however, the evaluation of x=a is forced, so x is bound to 2. This leads to a discrepancy between CBN in line 6, since x=a is evaluated again to produce 3, but CBNeed has already locked in x=2. CBR exhibits similar behavior as CBN here, while under CBV

Type	A, B	$::=$	Π	*(primitive type)*
		$\mid$	$(\overline{A}) \to B$	*(procedure type)*
TEnv	Γ, Δ	$::=$	$\bullet$	*(empty type environment)*
		$\mid$	$\Gamma, x : A$	*(extend with "x has type A")*

Figure 10: The type language for Base programs.

`c` is bound to `1`, as `x` hasn't been directly modified.

On line `7`, `x` is directly incremented, so CBV binds `x` to `2`. CBR binds `a` to `4`, since `x` points to `a`, while CBN forces the thunk pointed to by `x` to bind `x` to `4`. CBNeed refers to the already-forced `x`, binding it to `3`.

On line `8`↝`10`, `f` returns. CBR uniquely binds `a` to `1`, since `x` was modified, leading `a` to change indirectly. Note that `f` returns the same value for CBR and CBN otherwise, differing only in the binding for `a`. At line `11`, the discrepancy in the binding for `a` thus produces distinct values for `r` under CBR and CBN as well.

A Static Typing

The type behavior of a program generally does not depend on the parameter passing strategy. However, it can still be insightful to see a (static) type system for an imperative language, so we will describe one for Base in this appendix. Figure 10 provides a type language for Base.

We can always identify types for expressions and procedure bodies, as is depicted in the first half of Figure 11. However, we cannot so easily do the same for statements or programs, as in general they don't have a type we can assign them. We follow Cardelli [2004, Section 4] in that we define a judgment for well-typed statements/programs in place of assigning them, say, the unit type.

$$\Gamma \vdash _ \dashv \Delta \ : \ \mathsf{TEnv} \times (\mathsf{Stmt} \cup \mathsf{Prog}) \times \mathsf{TEnv}$$

This judgment is analogous to the evaluation judgment $\rho \vdash _ \dashv \sigma$: it takes a type environment and a statement/program, producing a new type environment that may have been populated with more bindings. The notation differs slightly from Cardelli [2004, Section 4], but is similar in intent. The second half of Figure 11 provides the corresponding rule system.

References

Luca Cardelli. 2004. "Type Systems." In: *CRC Handbook of Computer Science and Engineering*. (2nd ed.). CRC Press. Chap. 97.

Hanne Riis Nielson and Flemming Nielson. 1992. *Semantics With Applications: A Formal Introduction*. John Wiley & Sons, Inc., USA.

$\boxed{\Gamma \vdash _ : A}$ *(in the type context* Γ*, an expression/body has type* A*)*

$$\frac{x{:}A \in \Gamma}{\Gamma \vdash x : A}\ \text{Type-Var} \qquad \frac{c \text{ has type } \Pi}{\Gamma \vdash c : \Pi}\ \text{Type-Con}$$

$$\frac{\oplus \text{ has type } \Pi_1 \times \Pi_2 \to \Pi \qquad \Gamma \vdash e_1 : \Pi_1 \qquad \Gamma \vdash e_2 : \Pi_2}{\Gamma \vdash e_1 \oplus e_2 : \Pi}\ \text{Type-Bin}$$

$$\frac{\Gamma \vdash f : (A_1, \ldots, A_n) \to B \qquad (\Gamma \vdash e_i : A_i)^{i \in 1..n}}{\Gamma \vdash f(e_1, \ldots, e_n) : B}\ \text{Type-Inv}$$

$$\frac{(\Gamma_{i-1} \vdash s_i \dashv \Gamma_i)^{i \in 1..n} \qquad \Gamma_n \vdash e : A}{\Gamma_0 \vdash \{s_1; \ \ldots; \ s_n; \ \texttt{return}\ e;\} : A}\ \text{Type-Ret}$$

$\boxed{\Gamma \vdash _ \dashv \Delta}$ *(in the type context* Γ*, a statement/program is well-typed, producing* Δ*)*

$$\frac{}{\Gamma \vdash x : \Pi \dashv \Gamma, x{:}\Pi}\ \text{Type-Decl} \qquad \frac{x{:}A \in \Gamma \qquad \Gamma \vdash e : A}{\Gamma \vdash x = e \dashv \Gamma}\ \text{Type-Assign}$$

$$\frac{\Gamma, \overline{x{:}\Pi} \vdash Q : A}{\Gamma \vdash \texttt{proc}\ f(\overline{x : \Pi})Q \dashv \Gamma, f{:}(\overline{\Pi}) \to A}\ \text{Type-Proc} \qquad \frac{\Gamma \vdash P \dashv \Delta}{\Gamma \vdash P \dashv \Gamma}\ \text{Type-Internal}$$

$$\frac{(\Gamma_{i-1} \vdash s_i \dashv \Gamma_i)^{i \in 1..n}}{\Gamma \vdash \{s_1; \ \ldots; \ s_n\} \dashv \Gamma_n}\ \text{Type-Prog}$$

Figure 11: Typing rules for Base. Rules Type-Con and Type-Bin assume that the type of a constant/binary operation can be retrieved from an external source. `5` has type `Int` and `_&&_` has type `Bool`×`Bool` → `Bool`, for example. Note that rule Type-Internal does not restrict the domain of Δ to the domain of Γ, unlike the corresponding rule in Figure 4, for instance. This is since the type of an already declared variable cannot suddenly change within a nested block.